Topic: What Does a CNN See? | Visualizing CNN Filters and Feature Maps

➢ Introduction

When you train a CNN on images cats, cars, dogs it starts to automatically **learn features** from the images.

But it's not like a human seeing a "cat" as a whole.

Instead, CNNs learn **layers of features**, from very simple to very complex.

So, understanding "what CNN sees" means:

- Looking **inside** the network
- Observing **filters (kernels)** and **feature maps (activations)**

This helps us interpret what each layer learns and how images are represented.

➢ Concept 1: Filters (or Kernels)

Each convolutional layer in CNN has **filters** (small matrices, e.g. 3×3 or 5×5) that learn *patterns* from the input image.

Example:

- Early layers → detect *edges* (horizontal, vertical, diagonal)
- Middle layers → detect *textures*, *corners*, *color blobs*
- Deep layers → detect *high-level features* (eyes, faces, wheels, etc.)

The filters are the **eyes** of CNN — they look for specific visual cues in the image.

➢ Concept 2: Feature Maps (or Activation Maps)

When you pass an image through the CNN, each filter produces an **output image** showing *where* that feature is found.

That output is called a **feature map** (or activation map).

So, a **feature map tells us how strongly a certain feature appears and where** in the image it occurs.

➢ Layer-wise Understanding: What CNN Learns at Each Stage

Let's imagine feeding a **cat image** to a CNN like VGG16:

Layer	What It Sees	What It Learns
Layer 1–2 (Early layers)	Pixels, edges, corners, gradients	Basic features — shapes and lines
Layer 3–5 (Mid layers)	Patterns, textures, simple shapes	Fur textures, eyes, curves

Layer 6+ (Deep layers)	Object parts, complex shapes	Cat faces, tails, or full body
Fully Connected Layer	Entire object	Final classification (e.g., "cat")

So, early layers are like "seeing pixels," while deeper layers are like "seeing objects."

➢ How to Visualize Filters

You can visualize what each filter has learned by accessing the weights of a CNN layer.

Example (using Keras):

```
from tensorflow.keras.applications import VGG16
import matplotlib.pyplot as plt

# Load pre-trained model
model = VGG16(weights='imagenet', include_top=False)

# Get the first convolutional layer
filters, biases = model.layers[1].get_weights()

print(filters.shape)
# Output: (3, 3, 3, 64) → 64 filters of size 3x3x3 (RGB)

# Normalize filter values between 0 and 1 for visualization
f_min, f_max = filters.min(), filters.max()
filters = (filters - f_min) / (f_max - f_min)

# Plot first 6 filters
n_filters = 6
for i in range(n_filters):
    f = filters[:, :, :, i]
    plt.subplot(1, n_filters, i+1)
    plt.imshow(f)
    plt.axis('off')
plt.show()
```

Each filter image represents what pattern the CNN is focusing on in that layer.

➢ How to Visualize Feature Maps

Feature maps show *how an input image activates filters* in different layers.

Here's an example:

```
from tensorflow.keras.models import Model
import numpy as np
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.vgg16 import preprocess_input

# Load the model
model = VGG16(weights='imagenet', include_top=False)

# Select the layers to visualize
layer_outputs = [layer.output for layer in model.layers[:8]]
activation_model = Model(inputs=model.input, outputs=layer_outputs)

# Prepare input image
img_path = 'cat.jpg'
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

# Get feature maps
activations = activation_model.predict(x)

# Visualize first layer activations
first_layer_activation = activations[0]
print(first_layer_activation.shape)  # e.g. (1, 224, 224, 64)

# Plot first few feature maps
for i in range(6):
   plt.subplot(1, 6, i+1)
   plt.imshow(first_layer_activation[0, :, :, i], cmap='viridis')
   plt.axis('off')
plt.show()
```

Each map shows *where the CNN focuses* for that filter.

➢ Why Visualization Matters

Purpose	Explanation

1. Interpretability	Understand how the CNN detects and processes patterns.
2. Debugging	See if CNN focuses on relevant parts of an image (e.g., face, not background).
3. Model Trust	Helps ensure the model isn't learning wrong correlations.
4. Explainability	Makes the "black box" of deep learning more transparent.

➢ Typical Observations in Visualization

- **Early layers**: high-resolution, small edges or gradients
- **Middle layers**: patterns (like textures, fur, leaves)
- **Deep layers**: complex object parts (like eyes, wheels)
- **Final layers**: entire object representations

Each deeper layer combines simple features into complex ones — just like the human visual system (the visual cortex).

➢ Connection to Human Brain

CNNs are inspired by how the **visual cortex** works:

- Early neurons detect edges.
- Later neurons detect shapes.
- The deepest neurons detect whole objects.

So visualizing CNN layers gives insight into how our own vision might process information step by step.

➢ Key Takeaways

Concept	Explanation
Filter (Kernel)	Learns to detect specific features (edges, colors, patterns).
Feature Map (Activation)	Shows *where* those features appear in the image.
Early Layers	Detect simple features like edges or corners.
Deep Layers	Detect complex features like object parts.
Visualization Tools	Matplotlib, TensorBoard, Grad-CAM, etc.
Goal	To interpret what CNNs learn and make them explainable.

➢ In Simple Words

CNNs don't "see" the world like humans do.

They see **patterns**, **edges**, and **textures**, layer by layer building from simple shapes to full objects.

Visualization helps you peek *inside their mind* and understand how they make predictions.